

Data Structures – Unit 04

Bucharest University of Economic Studies

Linear structures, Simple chained lists



Agenda

- Characteristics
- Internal representation: graphical/analytical
- Allocation/Deallocation of memory
- Access to elements
- Linked lists operations
- Efficiency indicators
- Conversions
- Generic lists



Linked lists

- A collection of elements called nodes;
- Builds an order relation based on how each node is positioned when inserted in the list;
- Elements of same type – homogeneous structure;
- List's length – the number of nodes;
- Resembles the unidimensional arrays;
- HEAP allocated structure, managed by a pointer to the first element of the list called head.



Linked lists

- Linked lists are linear structures;
- Contains two different types of fields:
 - useful data – used for processing information;
 - link data – used to refer other similar elements, called nodes;
- The first and last node are called head and tail and could have special values in the link data field, based on the implementation:
 - **non-circular lists: NULL , 0;**
 - pointer to head for the tail (simple and double lists)
 - pointer to tail for the head (double lists)



Linked lists

Generic list with n elements:

$$A_1, A_2, A_3, \dots, A_{i-1}, A_i, A_{i+1}, \dots, A_n$$

Properties:

- list size = n ($n=0 \Rightarrow$ null list);
- first element = A_1 ;
- last element = A_n ;
- A_i succeeds A_{i-1} ($i \leq n$) – A_i is a **successor**;
- A_{i-1} precedes A_i ($i > 1$) – A_{i-1} is a **predecessor**;



Linked lists

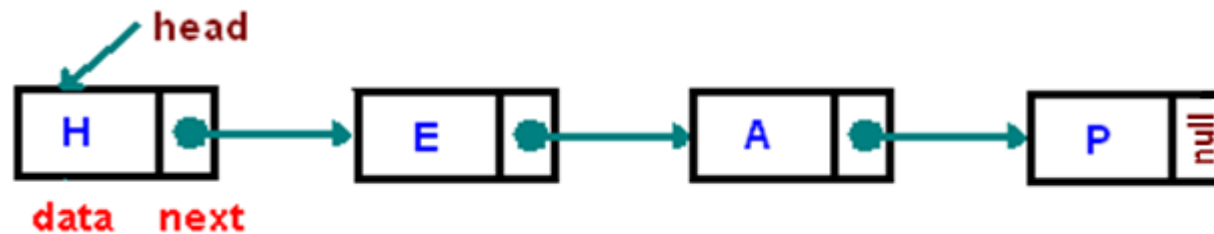
Implementation:

- By using pointer variables;
- Each node has two type of information:
 - Fields for useful data – for processing operations;
 - Fields for chaining nodes – pointers to other nodes in the structure.



Linked lists

Defining the structure of each node for a simple linked list:



```
struct Nod{  
    char  data;  
    Nod  *next;  
};
```



Linked lists

Declare a variable of type pointer to the Nod structure

```
Nod* prim= NULL;
```



Memory allocation in HEAP for one element of this type

```
prim = (Nod*)malloc(sizeof(Nod));
```



Disposing memory with free

```
free prim;
```



Linked lists

Declare a variable of type pointer to the Nod structure

```
Nod* prim= NULL;
```

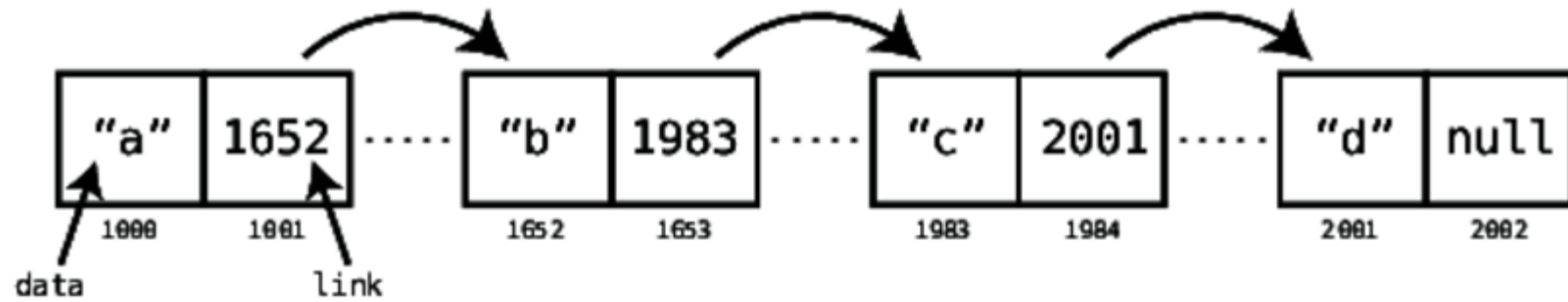
Memory allocation in HEAP for one element of this type

```
prim = new Nod;
```

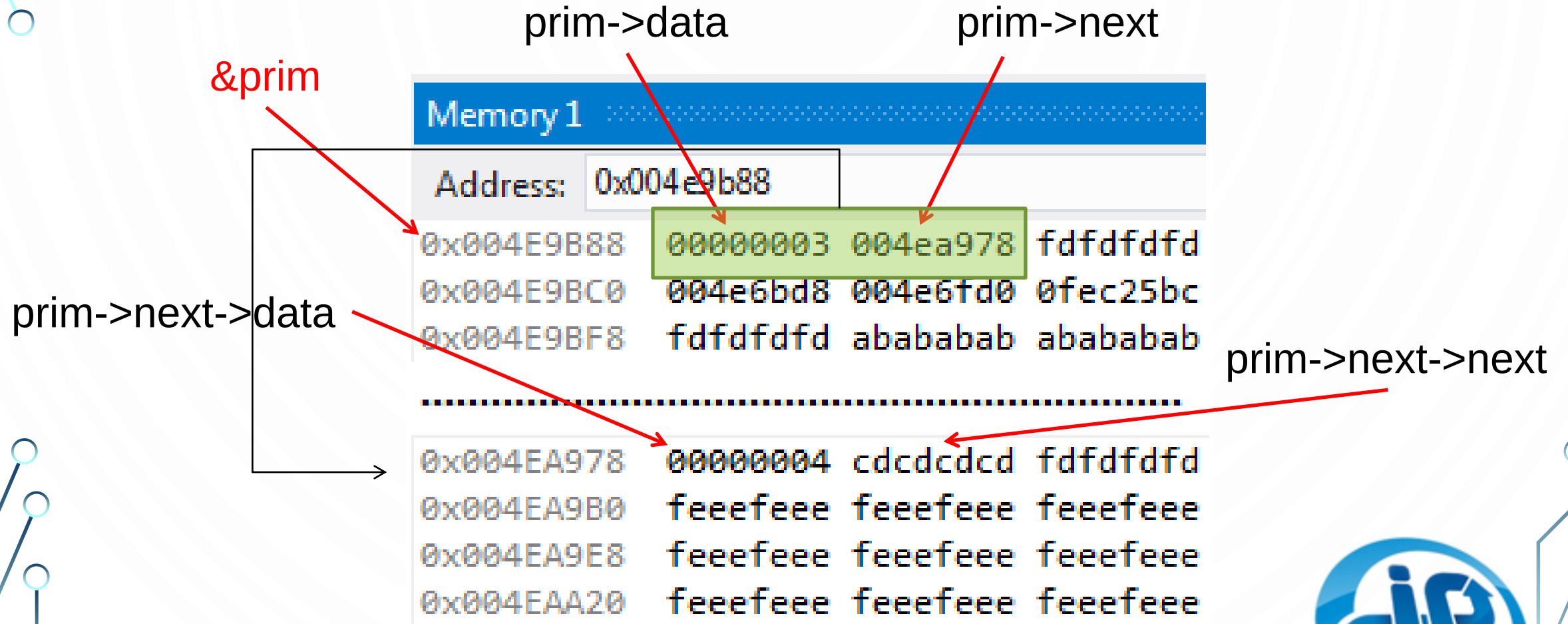
Disposing memory with delete

```
delete prim;
```





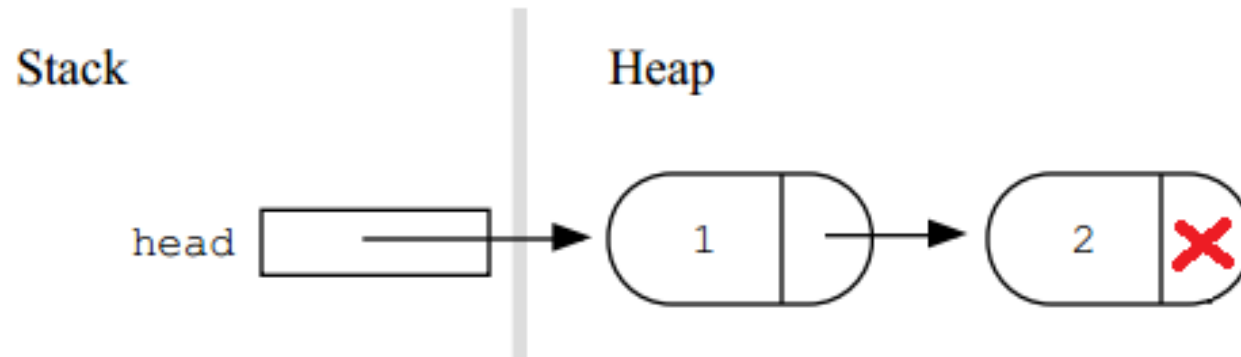
Linked lists



Linked lists

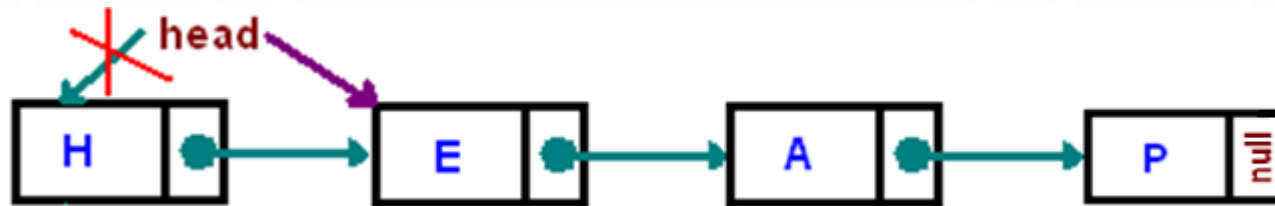
Memory Stack: 4 bytes

Memory Heap: 10 bytes



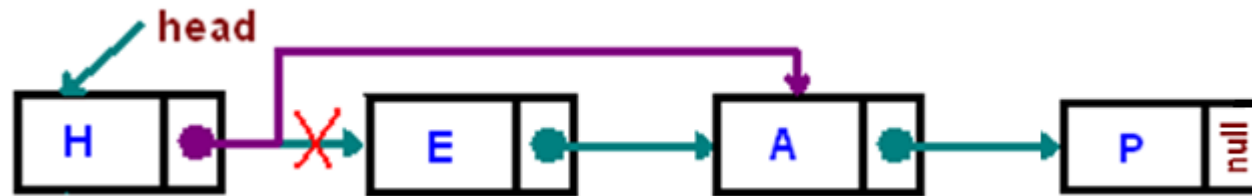
Linked lists

```
head = head->next;
```



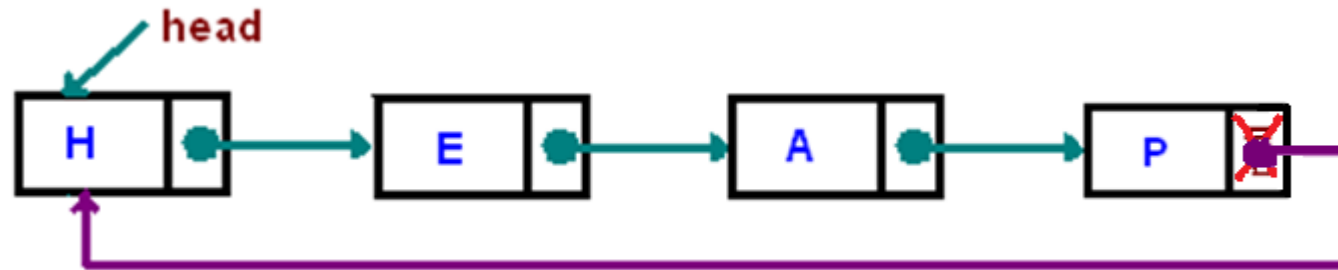
Linked lists

```
head->next = head->next->next;
```



Linked lists

head->next->next->next->next = head;



Linked lists vs. Arrays

Simple lists	Dynamic arrays	Static arrays
Variable number of elements, without knowing the size from the start		Fixed number of elements with memory reservation at the declaration stage
Elements allocated at random addresses from HEAP	Compact memory space allocated in the HEAP sector	Compact memory space in the STACK or DATA segment
Memory allocated at runtime		Memory allocated at the compiling time
Addressing each element at a time, with $O(n)$ complexity starting from the head	Addressing each element by its index in the structure added as an offset (index * sizeof(element_type) bytes) to the first element	
Additional memory needed for referring the structure	The structure is referred by using the name of the array which is also an address to the first element of the structure	

Linked lists vs. Arrays

	Lists	Arrays
Access to the i element	Sequential - $O(i)$	Direct - $O(1)$
Frequent operations for random access to memory		
Operations based on the current element (cursor)		
Structure resize		
Dummy nodes		



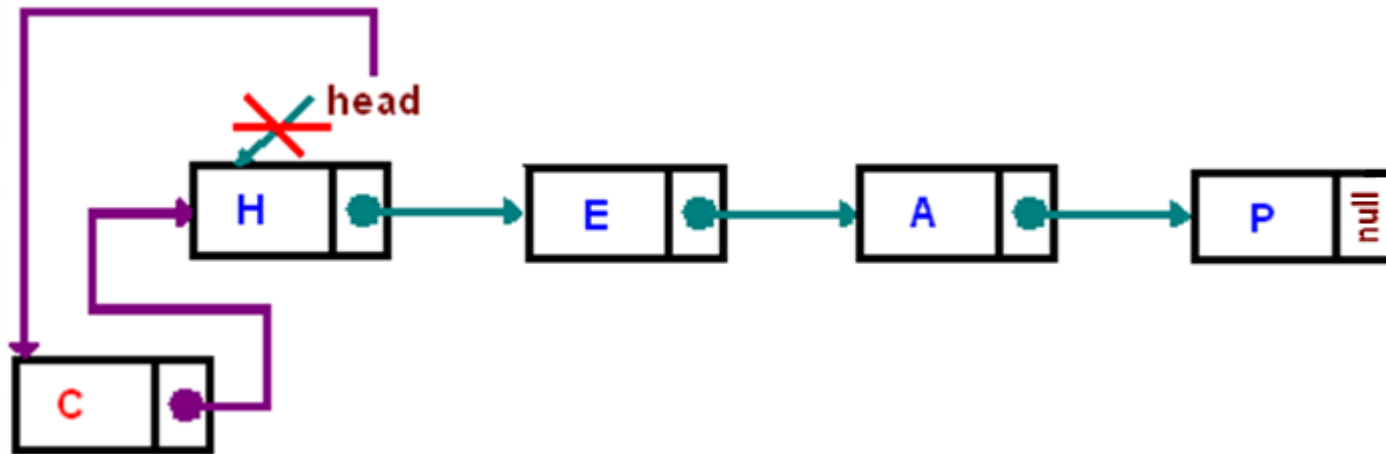
Linked lists operations

- Building a list by multiple insertions of a node;
- Traversing the structure;
- Searching for a node;
- Interchanging nodes;
- Deleting a node;
- Deleting a list;
- Sorting a list;
- Concatenating a list;
- Collating or merging a list.



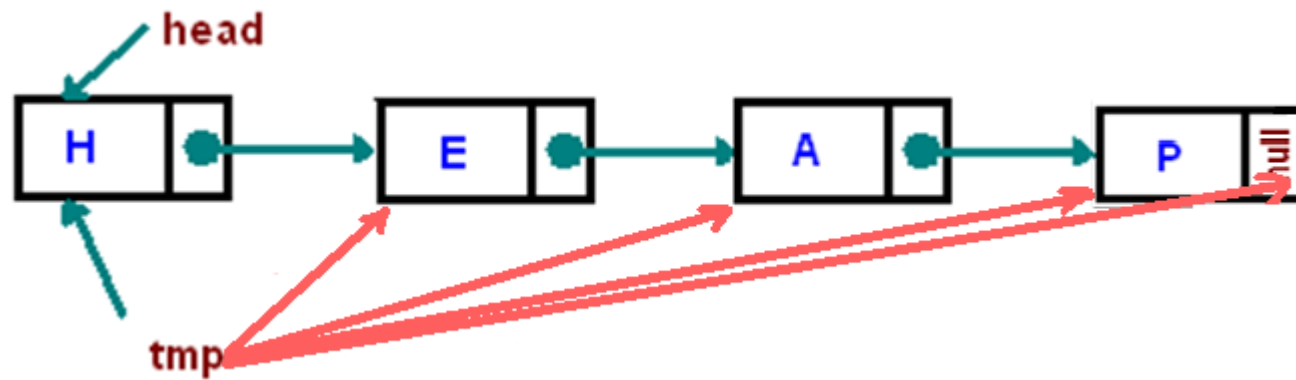
Simple Linked Lists

Head insertion...



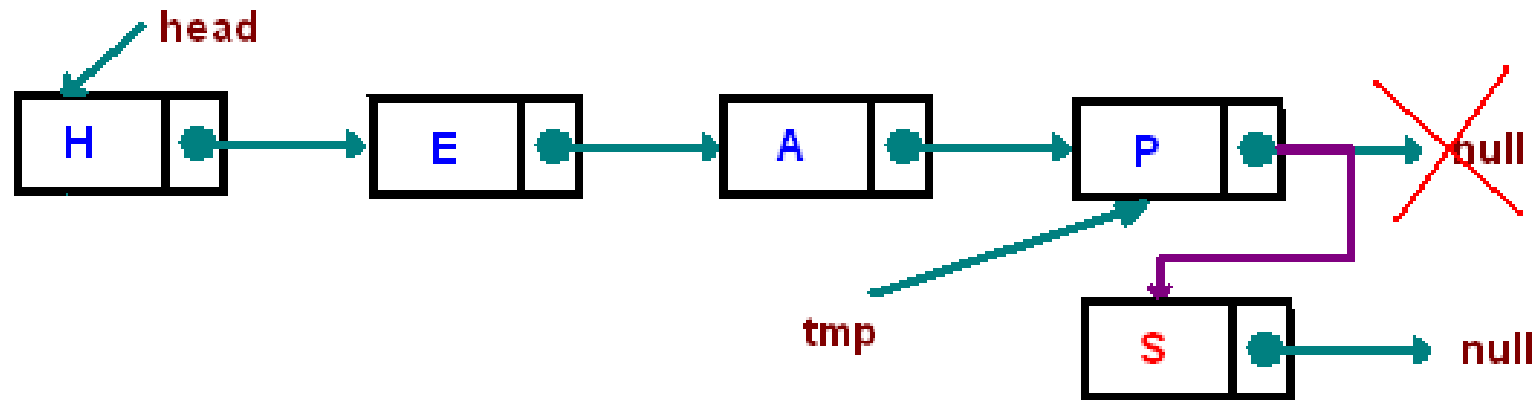
Simple Linked Lists

Traversing the list...



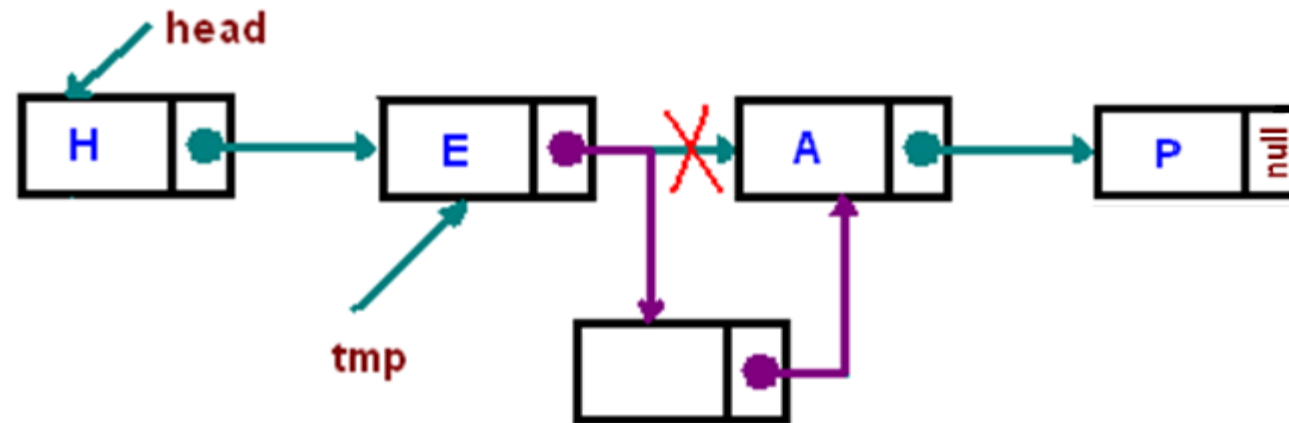
Simple Linked Lists

Tail insertion ...



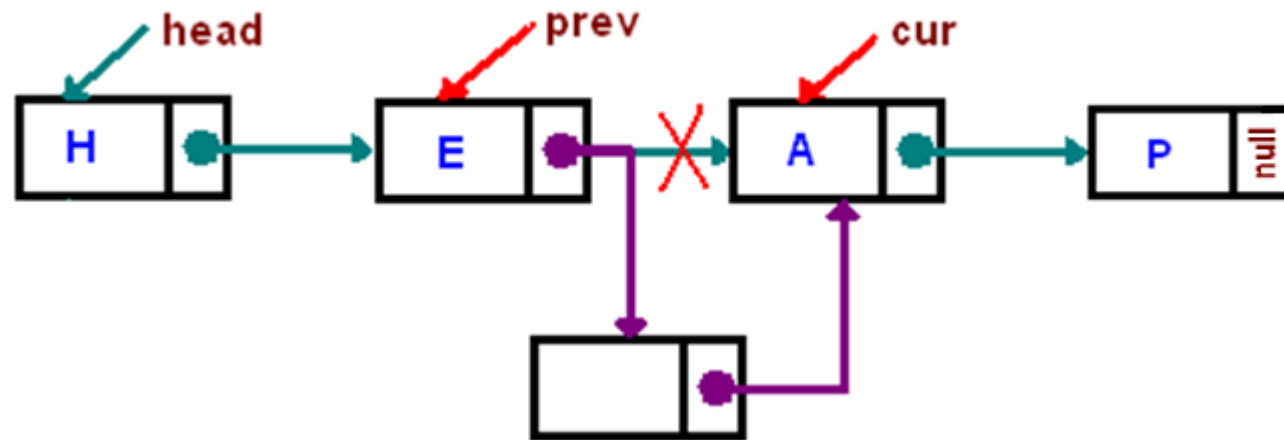
Simple Linked Lists

Insertion after a specified node ...



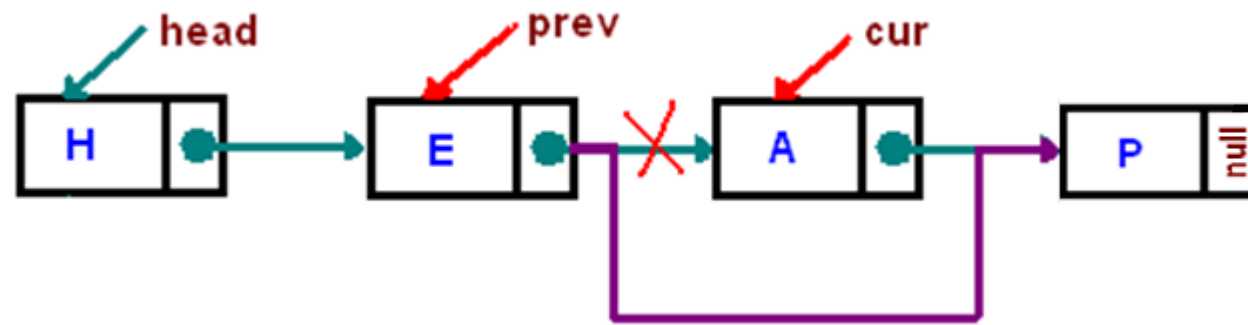
Simple Linked Lists

Insertion before a specified key ...



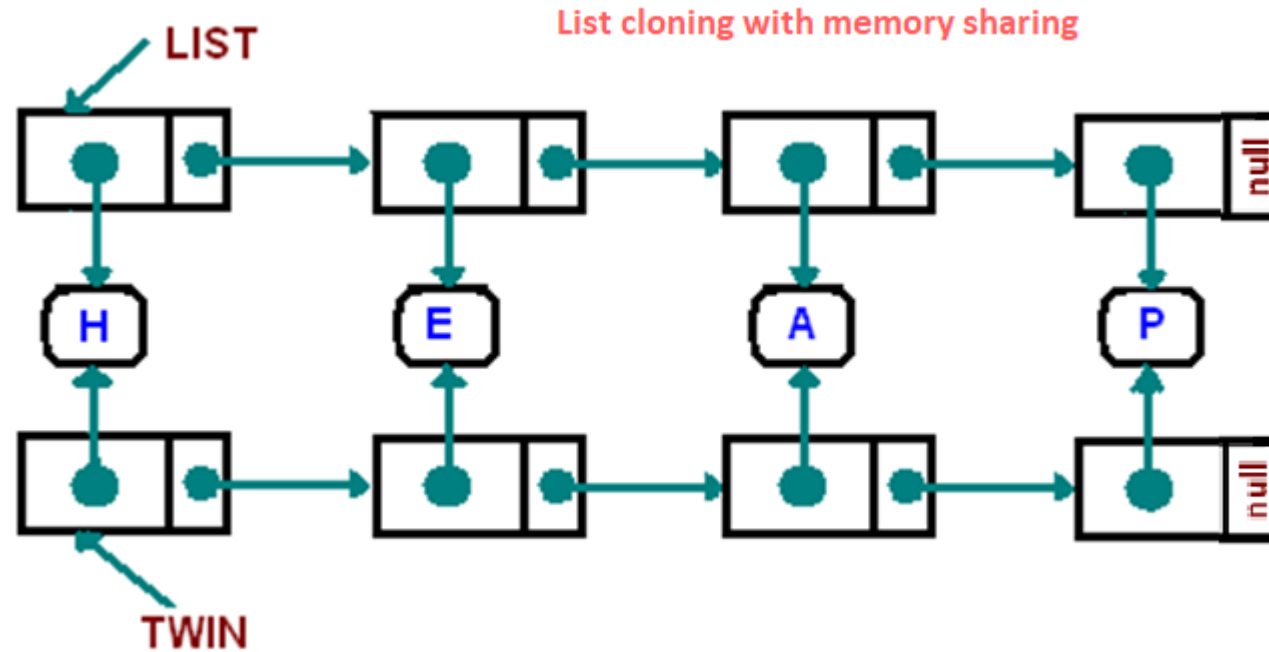
Simple Linked Lists

Deleting a node ...



Simple Linked Lists

Clone In-place



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

